Homework #5

December 9, 2020

Chun-Yuan (Scott) Chiu

chunyuac@andrew.cmu.edu

1.

(a)

Fraction of successes, as calculated below, is 11.78%.

[1]:
```
from sklearn.model_selection import train_test_split
import pandas as pd
import numpy as np

X = pd.read_csv('marketing.csv')
X = pd.get_dummies(X, columns=['job', 'marital', 'education', 'default',␣
 ↪'housing', 'loan'])
y = np.where(X.pop('y')=='yes', 1, 0)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.33,␣
 ↪random_state=1)
y_train.sum()/len(y_train)
```

[1]:
```
0.11782377603908752
```

(b)

Here we train a logistic regression model, serialize the trained model and save it locally to prevent retraining.

[2]:
```
from sklearn.linear_model import LogisticRegression
import pickle

clf = LogisticRegression(penalty='none').fit(X_train, y_train)

with open('clf.pkl', 'wb') as f:
    pickle.dump(clf, f)
```

```
with open('clf.pkl', 'rb') as f:
    clf = pickle.load(f)
```

The misclassification rate calculated below is 11.55%.

[2]:
```
from sklearn.metrics import accuracy_score
import pickle

with open('clf.pkl', 'rb') as f:
    clf = pickle.load(f)

y_pred = clf.predict(X_test)
1 - accuracy_score(y_test, y_pred)
```

[2]:
```
0.11548257372654158
```

(c)

A classifier to always say 'no' will have misclassification rate 11.53%, not much different from that of the logistic regression.

[3]:
```
y_pred = np.zeros_like(y_test)
1 - accuracy_score(y_test, y_pred)
```

[3]:
```
0.11528150134048254
```

(d)

Only 11.53% out of all clients in the test data said yes. This is the sample proportion, also the MLE estimate of the fraction. If we randomly picked 1000 client to call, this is the fraction we can expect to say 'yes'.

Selecting the first 1000 clients who have the largest probabilities to say 'yes' according to the logistic regression, the fraction increases to 29.1% (more than doubled).

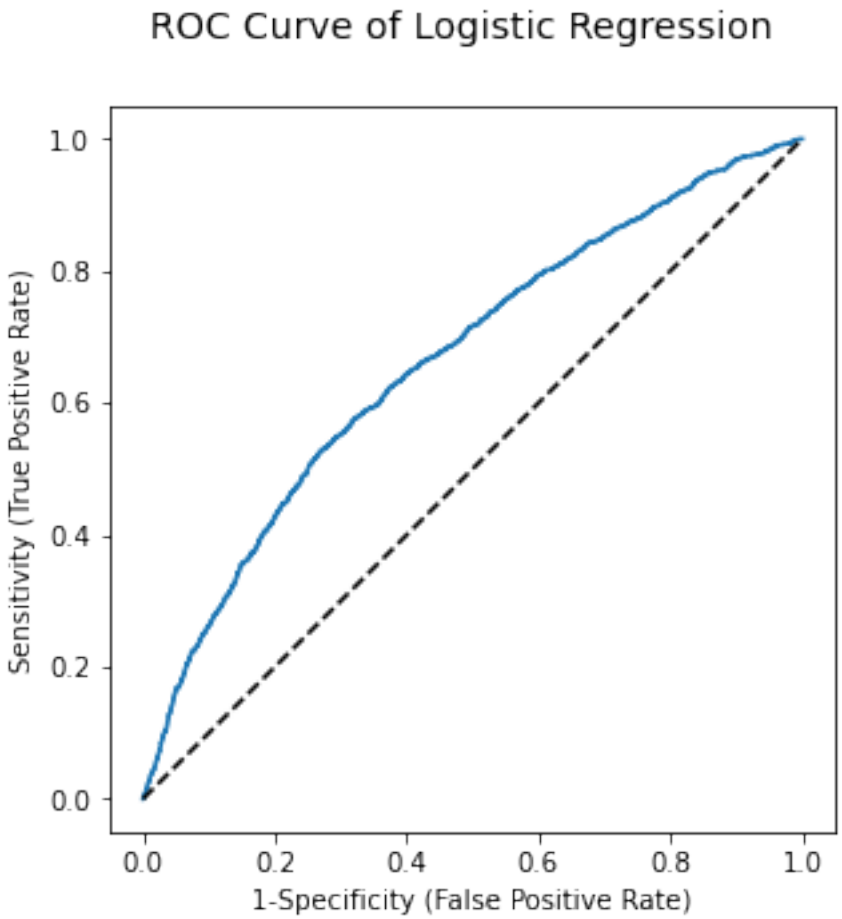
[4]:
```
predict_proba_test = clf.predict_proba(X_test)[:, 1]
bestIdxSet = predict_proba_test.argsort()[-1000:]

y_select = y_test[bestIdxSet]

print('Fraction of clients who say yes from random selection: ', y_test.sum()/
 ↪len(y_test))
print('Fraction of clients who say yes from logistic regression: ', y_select.
 ↪sum()/len(y_select))
```

```
Fraction of clients who say yes from random selection:  0.11528150134048257
Fraction of clients who say yes from logistic regression:  0.291
```

(e)

Below is a the ROC curve plot and a convenient `plot_roc` function for later reuse.

[5]:
```
import matplotlib.pyplot as plt
from matplotlib import cm
from sklearn.metrics import roc_curve
from pandas import DataFrame

fpr, tpr, _ = roc_curve(y_test, predict_proba_test)

def plot_roc(spec, sens):
    fig, ax = plt.subplots(1, 1, figsize=(5, 5))
    fig.suptitle('ROC Curve of Logistic Regression', fontsize=14)
    ax.plot(1-spec, sens)
    ax.plot([0, 1], [0, 1], 'k--')
    ax.set( aspect=1,
            xlabel='1-Specificity (False Positive Rate)',
            ylabel='Sensitivity (True Positive Rate)')
    return ax

plot_roc(1-fpr, tpr)
plt.show()
```

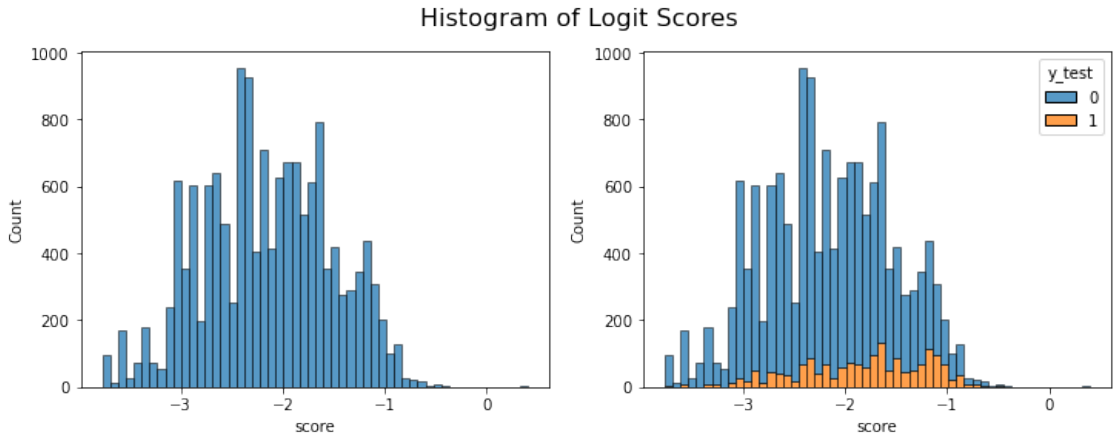


2.

(a)

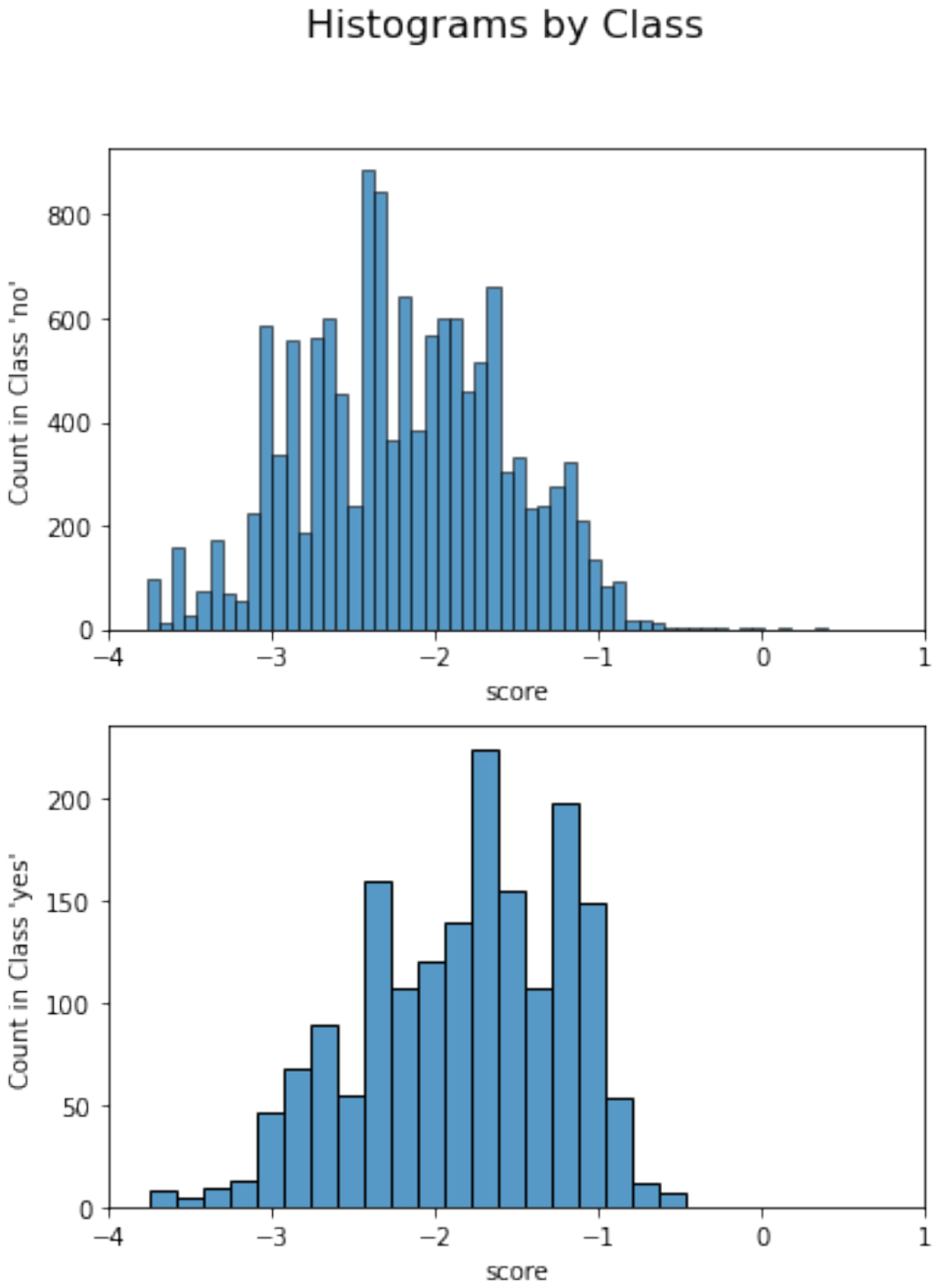
[6]:
```
import matplotlib.pyplot as plt
import seaborn as sns

pn, pp = clf.predict_proba(X_test).T
scores = np.log(pp/pn)

df = DataFrame({'score': scores, 'y_test': y_test})

fig, ax = plt.subplots(1, 2, figsize=(12, 4))

sns.histplot(scores, ax=ax[0])
sns.histplot(x='score', hue='y_test', data=df, multiple='stack', ax=ax[1])
```

```
ax[0].set(xlabel='score')
plt.suptitle('Histogram of Logit Scores', fontsize=16)
plt.show()
```



(b)

[7]:
```
fig, ax = plt.subplots(2, 1, figsize=(6, 8))

sns.histplot(x='score', data=df[df['y_test']==0], ax=ax[0])
sns.histplot(x='score', data=df[df['y_test']==1], ax=ax[1])
ax[0].set(xlim=(-4, 1), ylabel="Count in Class 'no'")
ax[1].set(xlim=(-4, 1), ylabel="Count in Class 'yes'")
plt.suptitle('Histograms by Class', fontsize=16)
plt.show()
```


Histograms by Class
Count in Class 'no'
800
600
400
200
0
−4
−3
−2
−1
0
1
score
Count in Class 'yes'
200
150
100
50
0
−4
−3
−2
−1
0
1
score

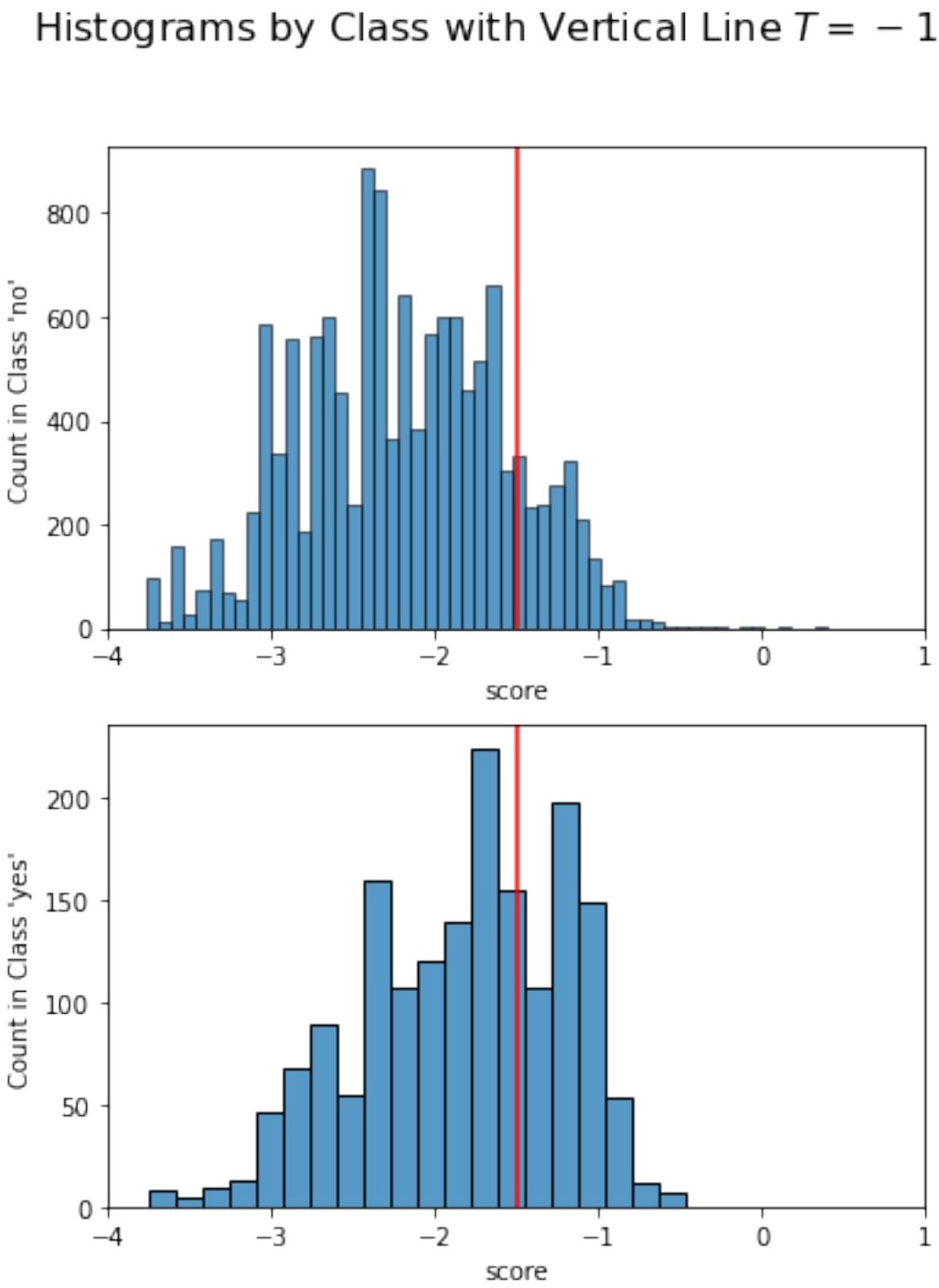
(c)

We can write

$$\begin{aligned} TPR(T) &= \mathbb{P}(\text{score} > T \mid \text{class 'yes'}) = \int_T^\infty f_1(x)\,dx, \\ FPR(T) &= 1 - \text{specificity} \\ &= 1 - \mathbb{P}(\text{score} < T \mid \text{class 'no'}) \\ &= 1 - \mathbb{P}(\text{score} > T \mid \text{class 'no'}) \\ &= \int_T^\infty f_0(x)\,dx. \end{aligned}$$

In the below figure, TPR corresponds to the blue area on the right of the vertical line on the class 'yes' histogram, while FPR corresponds to the blue area on the right of the vertical line on the class 'no' histogram.

[8]:
```
fig, ax = plt.subplots(2, 1, figsize=(6, 8))

sns.histplot(x='score', data=df[df['y_test']==0], ax=ax[0])
sns.histplot(x='score', data=df[df['y_test']==1], ax=ax[1])
ax[0].set(xlim=(-4, 1), ylabel="Count in Class 'no'")
ax[1].set(xlim=(-4, 1), ylabel="Count in Class 'yes'")
ax[0].axvline(x=-1.5, color='r')
ax[1].axvline(x=-1.5, color='r')
plt.suptitle('Histograms by Class with Vertical Line $T=-1.5$', fontsize=16)
plt.show()
```


Histograms by Class with Vertical Line $T = -1.5$
Count in Class 'no'
800
600
400
200
0
−4
−3
−2
−1
0
1
score
Count in Class 'yes'
200
150
100
50
0
−4
−3
−2
−1
0
1
score

(d)

We know that the area under a curve with parametric form $(f(t), g(t))$ is $\int_{t_0}^{t_f} g(t)f'(t)\,dt$. Now plug in $f(t) = FPR(t)$, $g(t) = TPR(t)$, $t_0 = \infty$ and $t_f = -\infty$ to get

$$\begin{aligned} AUC &= \int_{t_0}^{t_f} g(t)f'(t)\,dt \\ &= \int_{\infty}^{-\infty} TPR(t)FPR'(t)\,dt. \end{aligned}$$

(e)

Note that

$$FPR'(T) = \frac{d}{dT}\int_{T}^{\infty} f_0(x)\,dx = -f_0(T).$$

Plug this into the above AUC integral to get

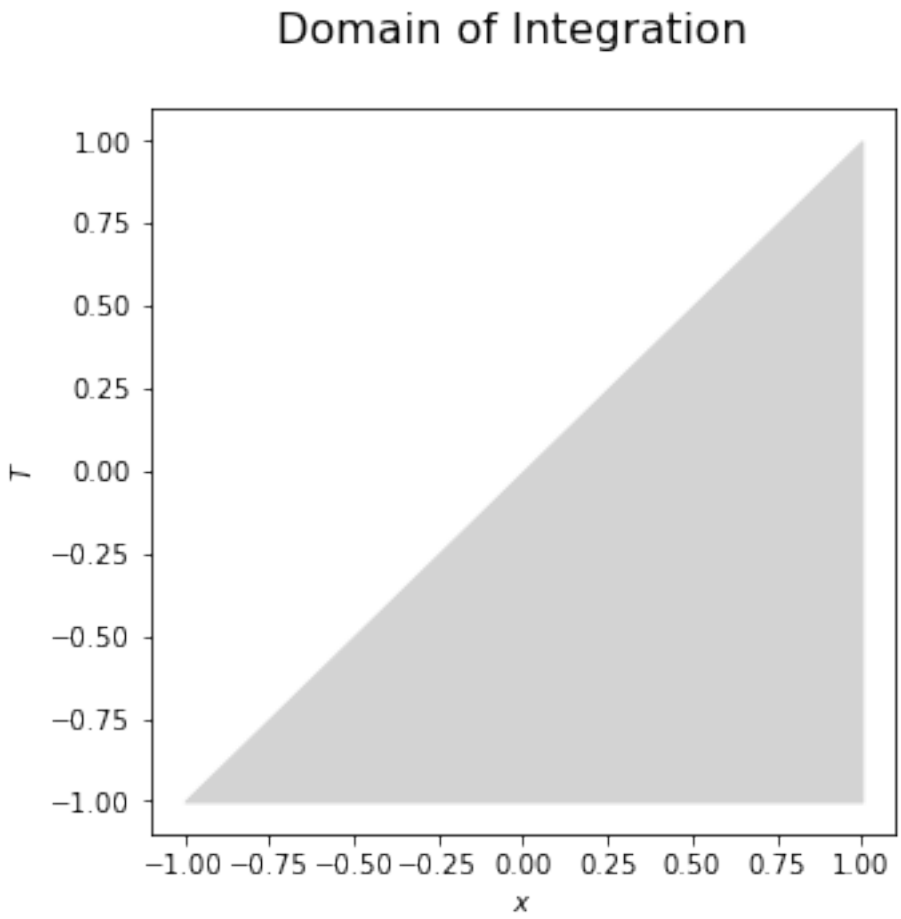
$$\begin{aligned} AUC &= \int_{\infty}^{-\infty} \left(\int_{T}^{\infty} f_1(x)\,dx\right)(-f_0(T))\,dT \\ &= \int_{-\infty}^{\infty}\int_{T}^{\infty} f_0(T)f_1(x)\,dx\,dT. \end{aligned}$$

Assuming that S_0 and S_1 are independent, $f_0(T)f_1(x)$ is simply their joint distribution, so this integral is the probability of (S_1, S_0) being in the region $\{(x,T) \in \mathbb{R}^2 \mid -\infty < T < \infty, T < x < \infty\} = \{(x,T) \in \mathbb{R}^2 \mid T < x\}$, which is equivalent to the $\mathbb{P}(S_1 > S_0)$. Thus we conclude

$$AUC = \mathbb{P}(S_1 > S_0).$$

Below is a plot of the domain of integration.

[9]:
```
import matplotlib.pyplot as plt
from pandas import DataFrame

fig, ax = plt.subplots(1, 1, figsize=(5, 5))
x = [-1, 1]
y = [-1, 1]
ax.fill_between(x, -1, y, color='lightgray')
ax.set(xlabel='$x$', ylabel='$T$')
plt.suptitle('Domain of Integration', fontsize=16)
plt.show()
```



3.

(a)

The array of logit scores is already computed in 2(a) (as `scores`). The `eval_estimate` implementation is straightforward from the definitions. The result with threshold -1 is printed in the end.

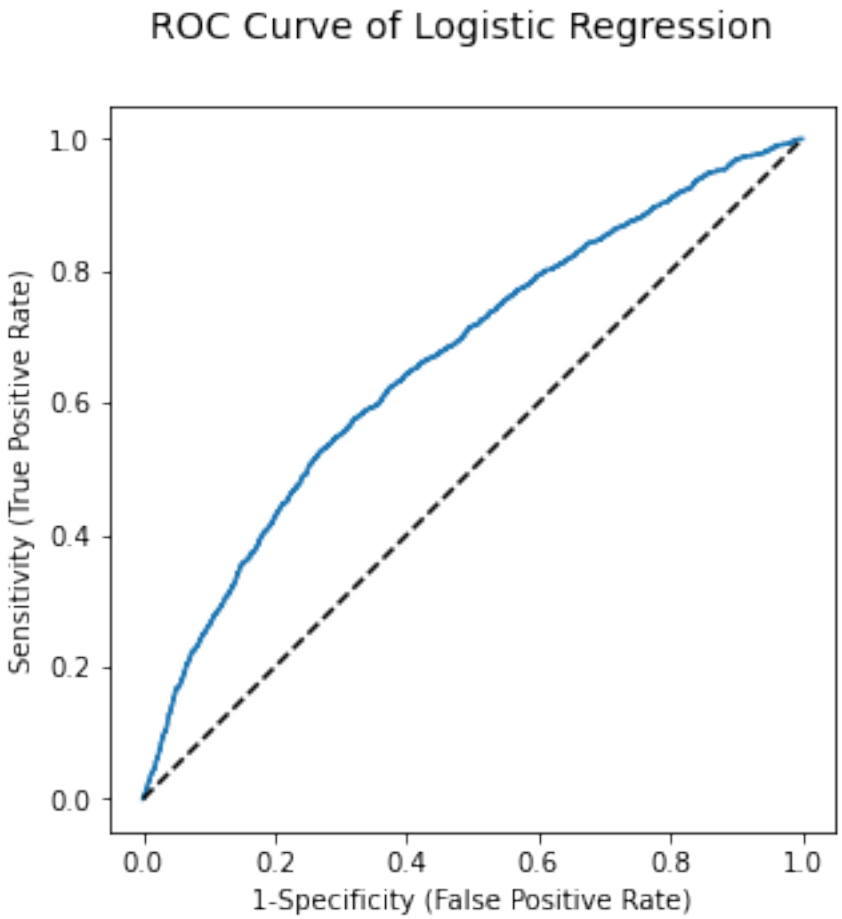
[10]:
```
import numpy as np

threshold = -1

def eval_estimate(estimate, truth, loss_FP=5, loss_FN=100):
    tp = (truth & estimate).sum()
    fp = ((1-truth) & estimate).sum()
    fn = (truth & (1-estimate)).sum()
    tn = ((1-truth) & (1-estimate)).sum()
```

```
    sens = tp/(tp + fn)
    spec = tn/(tn + fp)
    loss = fp*loss_FP + fn*loss_FN

    return (sens, spec, loss)

eval_estimate(scores > threshold, y_test)
```

[10]: (0.053488372093023255, 0.9808333333333333, 164065)

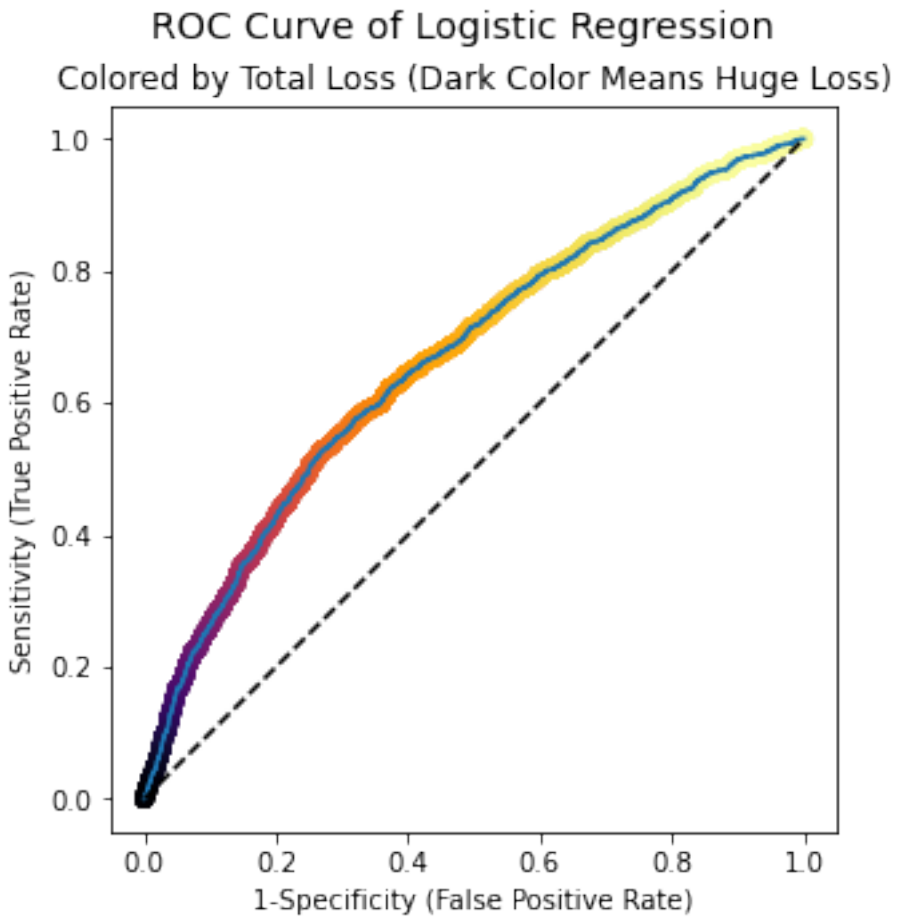
(b)

[11]:
```
unique_logits = np.sort(np.unique(scores))
midpts = (unique_logits[1:] + unique_logits[:-1])/2
sens, spec, loss = np.vectorize(lambda threshold: eval_estimate(scores >␣
 ↪threshold, np.array(y_test, dtype=bool)))(midpts)

plot_roc(spec, sens)
plt.show()
```



(c)

[12]:
```
ax = plot_roc(spec, sens)
ax.scatter(1-spec, sens, c=-loss, cmap=cm.inferno)
ax.set(title='Colored by Total Loss (Dark Color Means Huge Loss)')
plt.show()
```

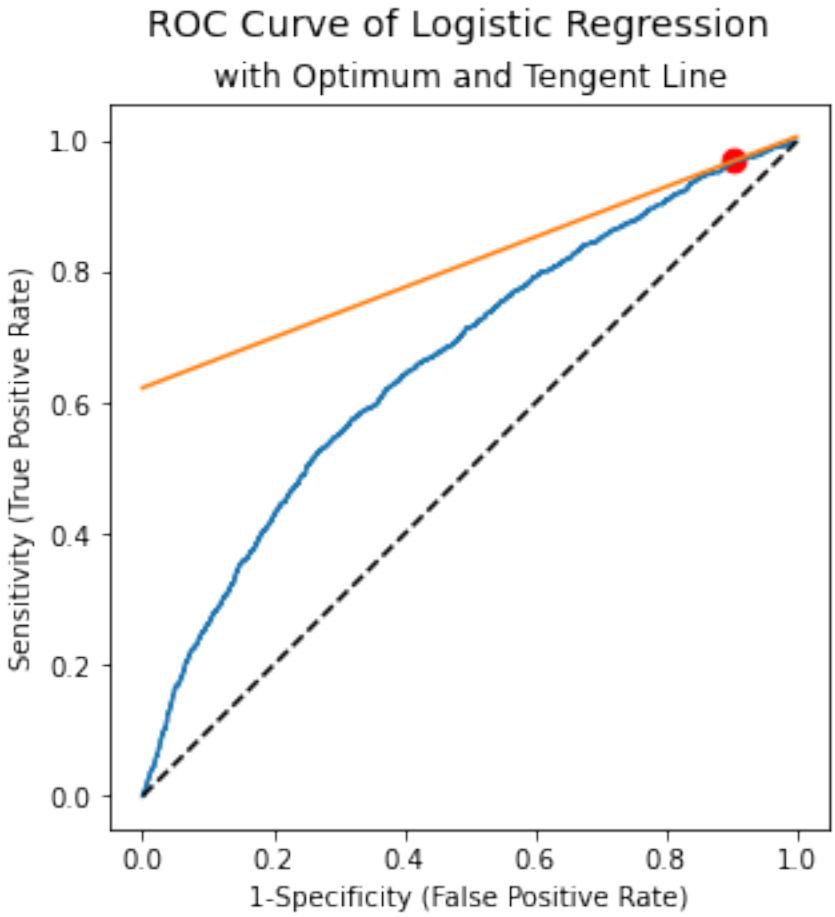


(d)

The red dot is the specificity and sensitivity that correspond to a choice of threshold that actually minimizes the loss in test data. We will call this threshold the empirical optimal threshold from now on. The line drawn as instructed is the tangent line of the ROC curve at the optimum.

[13]:
```
optimumIdx = np.argmin(loss)

x0 = 1-spec[optimumIdx]
y0 = sens[optimumIdx]

loss_FP = 5
loss_FN = 100
P = (y_test).sum()
N = (1 - y_test).sum()
slope = (N/P)*(loss_FP/loss_FN)
lineEq = lambda x: slope*(x-x0) + y0
```

```
ax = plot_roc(spec, sens)
ax.plot([0, 1], [lineEq(0), lineEq(1)])
ax.scatter([x0], [y0], color='red', s=75)
ax.set(title='with Optimum and Tengent Line')
plt.show()
```



(e)

i.

Define TN, FP, TP and FN in the test set to be the number of true negative, false positive, true positive and false negative observations, respectively. Then we have

$$\begin{aligned} FPR &= \frac{FP}{TN + FP}, \\ FNR &= \frac{FN}{TP + FN}, \\ N &= TN + FP, \\ P &= TP + FN. \end{aligned}$$

Thus we know $FP = FPR \cdot N$ and $FN = FNR \cdot P$ and hence the total loss is

$$FP \cdot L_{FP} + FN \cdot L_{FN} = FPR \cdot N \cdot L_{FP} + FNR \cdot P \cdot L_{FN}.$$

ii.

In 2(c) we have established that $FPR(T) = \int_T^\infty f_0(x)\,dx$. In a similar manner, we can write

$$FNR(T) = \mathbb{P}(\text{score} < T \mid \text{class 'yes'}) = \int_{-\infty}^{T} f_1(x)\,dx.$$

iii.

Taking derivatives of

$$FPR(T) = \int_T^\infty f_0(x)\,dx, \qquad FNR(T) = \int_{-\infty}^{T} f_1(x)\,dx,$$

we obtain $FPR'(T) = -f_0(T), FNR'(T) = f_1(T)$. Thus, the derivative of the total loss is

$$FPR'(T) \cdot N \cdot L_{FP} + FNR'(T) \cdot P \cdot L_{FN} = -f_0(T) \cdot N \cdot L_{FP} + f_1(T) \cdot P \cdot L_{FN}.$$

Set this to zero and isolate $f_1(T)/f_0(T)$ to obtain[1]

$$\frac{f_1(T)}{f_0(T)} = \frac{N}{P} \cdot \frac{L_{FP}}{L_{FN}}.$$

This is the formula we used in 3(d). The proof is complete.

[1] A parameteric form of the ROC curve is $(x(t), y(t)) = (FPR(t), TPR(t))$ so the slope of the tangent line is $dy/dx = (dy/dt)/(dx/dt) = TPR'(t)/FPR'(t) = f_1(t)/f_0(t)$.

(f)

The theoretical optimal threshold on the true probability, as derived in the previous homework, is $L_{FP}/(L_{FP}+L_{FN})$, which is $5/105 = 0.0476$ given our loss function here. We then apply the logit transformation to obtain the theoretical optimal threshold on the logit scores so that we can make use of the `eval_estimate` function implemented earlier. The losses using the theoretical and empirical optimal thresholds are reported below. They are different but close (with relative error 1.29%).

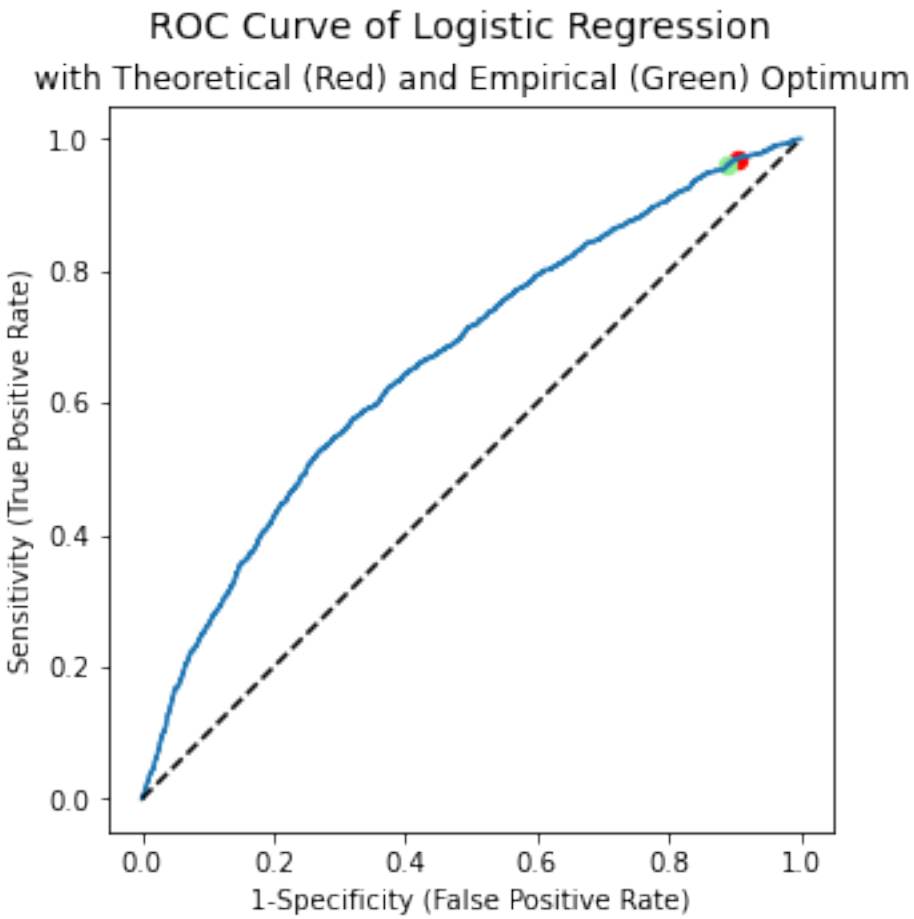
[15]:
```
threshold_prob = 5/105
threshold_logit = np.log(threshold_prob/(1 - threshold_prob))

sensBestTrueProb, specBestTrueProb, lossBestTrueProb = eval_estimate(scores >␣
↪threshold_logit, y_test)

print('Loss at theoretical optimal threshold: ', lossBestTrueProb)
print('Minimum loss found in 3(d):            ', np.min(loss))
```

```
Loss at theoretical optimal threshold:  65530
Minimum loss found in 3(d):             64695
```

(g)

We present the ROC curve with the theoretical (red) and empirical (green) optimum. They do not agree completely for many reasons. First, the logistic regression imposes a strong linearity assumption on the log odds of data. As this is most likely not the underlying truth, the empirical optimum obtained under this assumption can be suboptimal. Even if the assumption holds, the empirical optimum itself is random by nature. It is computed given the test data and fitted model, both of which are random. The randomness of the fitted model is from the training data. A different train-test split will lead to a (slightly) different model. Thus the empirical optimum is at best an estimate of the theoretical one. Given more data they might get closer but the probability of matching exactly is zero.

[16]:
```
optimumIdx = np.argmin(loss)

x0 = 1-spec[optimumIdx]
y0 = sens[optimumIdx]

x1 = 1-specBestTrueProb
y1 = sensBestTrueProb

ax = plot_roc(spec, sens)
ax.scatter([x0], [y0], color='red', s=40)
ax.scatter([x1], [y1], color='lightgreen', s=40)
ax.set(title='with Theoretical (Red) and Empirical (Green) Optimum')
plt.show()
```


ROC Curve of Logistic Regression
with Theoretical (Red) and Empirical (Green) Optimum
Sensitivity (True Positive Rate)
1-Specificity (False Positive Rate)
0.0
0.2
0.4
0.6
0.8
1.0